语音识别

语音识别

- 例程效果
- 代码讲解
  1. 导入必要的模块
  2. 定义全局变量和常量
  3. 定义 `AudioRecorderWithUpload` 类
     - 3.1 UI绘制方法
     - 3.2 网络和文件处理方法
     - 3.3 录音和语音识别方法
  4. 辅助函数
  5. 主程序逻辑
  6. 异常处理与资源释放

例程效果

本节例程源码文件名为 `voice_recognition.py`。

连接IDE，在IDE中运行代码，程序会自动连接WiFi，监听按键事件：按下按键开始录音，松开按键停止录音。随后，录音文件将被上传到百炼平台并转换为文字，识别结果显示在屏幕上。界面支持实时状态更新，如录音中、上传中和识别完成。



代码讲解

本节将详细解析 `voice_recognition.py` 代码的功能和实现逻辑，帮助用户理解如何实现音频录音、上传和语音识别的功能，包括UI绘制、网络连接和API交互。

1. 导入必要的模块

```
import os
from media.media import *
from media.display import *
from media.pyaudio import *
import media.wave as wave
import time
from ybUtils.YbKey import YbKey
import ujson as json
try:
    import urequests as requests
except ImportError:
    import requests
import libs.upload_image as upload_image
```

- `os`：用于文件系统操作，如目录检查和退出点。
- `media.media`，`media.display`，`media.pyaudio`，`media.wave`：用于音频录制、显示和媒体管理。
- `time`：用于时间控制和延时。
- `ybUtils.YbKey`：用于按键检测。
- `ujson`：用于JSON数据处理。
- `requests` (或 `urequests`)：用于发送HTTP请求。
- `libs.upload_image`：自定义模块，用于图像上传（此处用于音频上传逻辑）。

2. 定义全局变量和常量

```
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
UI_COLORS = { ... }
```

- `DISPLAY_WIDTH`，`DISPLAY_HEIGHT`：屏幕分辨率常量，分别为640x480。
- `UI_COLORS`：字典，定义界面颜色方案，用于绘制不同状态的UI元素，如背景、文本和警告颜色。

3. 定义 `AudioRecorderWithUpload` 类

```
class AudioRecorderWithUpload:
    def __init__(self, api_key, model_name="paraformer-v2", wifi_ssid="Yahboom",
wifi_key="yahboom890729", enable_ui=True):
```

- `AudioRecorderWithUpload` 类是程序的核心，负责音频录制、UI管理、网络连接和语音识别。
- 初始化参数包括API密钥、`model_name` (默认"paraformer-v2")、WiFi凭证和是否启用UI。
- 设置音频参数如格式、通道、采样率等，并初始化PyAudio和MediaManager。
- 如果启用UI，初始化显示器并创建图像对象。

3.1 UI绘制方法

- `draw_rounded_rect()`：绘制圆角矩形，用于UI卡片。
- `draw_loading_spinner()`：绘制加载动画。
- `draw_recording_indicator()`：绘制录音指示器。
- `draw_text_with_wrapping()`：绘制带换行的文本。
- `update_ui()`：更新整个界面，根据状态显示不同元素，如录音中或识别完成。

3.2 网络和文件处理方法

- `connect_wifi()`：连接WiFi网络，支持超时和UI反馈。
- `basename()`：获取文件路径的basename。
- `file_exists()`：检查文件是否存在。
- `get_upload_policy()`：获取上传凭证，支持重试。
- `upload_file_to_oss()`：上传文件到OSS，支持重试。
- `upload_audio_to_oss()`：上传音频文件。
- `ensure_dir()`：确保目录存在。

3.3 录音和语音识别方法

- `record_with_button_and_upload()`：使用按键控制录音，完成后上传文件。
- `record_and_upload()`：固定时长录音并上传。
- `record_with_button_and_speech_to_text()`：按键录音并进行语音识别。
- `record_and_speech_to_text()`：固定时长录音并识别。
- `speech_to_text_from_oss()`：从OSS URL进行语音转文字，包括任务提交和状态检查。
- `submit_voice_task()`：提交语音识别任务。
- `check_task_status()`：检查任务状态。
- `download_transcription_result()`：下载转录结果。
- `stop()`：停止录音。
- `_save_to_wav()`：保存音频文件为WAV格式。

4. 辅助函数

- `ensure_dir()`：在主程序中用于创建保存目录。

5. 主程序逻辑

```
if __name__ == "__main__":
    os.exitpoint(os.EXITPOINT_ENABLE)

    api_key = ""
    model_name = "paraformer-v2"
    save_path = "/data/audio/"
    wifi_ssid = ""
    wifi_key = ""

    ensure_dir(save_path)

    recorder = AudioRecorderWithUpload(api_key, model_name, wifi_ssid, wifi_key,
enable_ui=True)
    key = YbKey()

    try:
        while True:
```

```
            success, result = recorder.record_with_button_and_speech_to_text(
                save_path + 'button_audio.wav',
                key
            )

            if success:
                print(f"✓ 语音识别成功!")
                print(f"识别结果: {result}")
            else:
                print(f"✗ 语音识别失败: {result}")
    except Exception as e:
        print(f"操作失败: {e}")
    print("程序结束")
```

- 初始化目录、录音器和按键对象。
- 进入循环，监听按键：按下开始录音，松开停止并进行语音识别。
- 根据结果更新UI和控制台输出。
- 异常处理：捕获错误并更新UI状态。

6. 异常处理与资源释放

- 在方法中（如 `record_with_button_and_upload()`）使用 `finally` 块确保停止录音流。
- 主程序使用 `try-except` 捕获异常并打印错误。